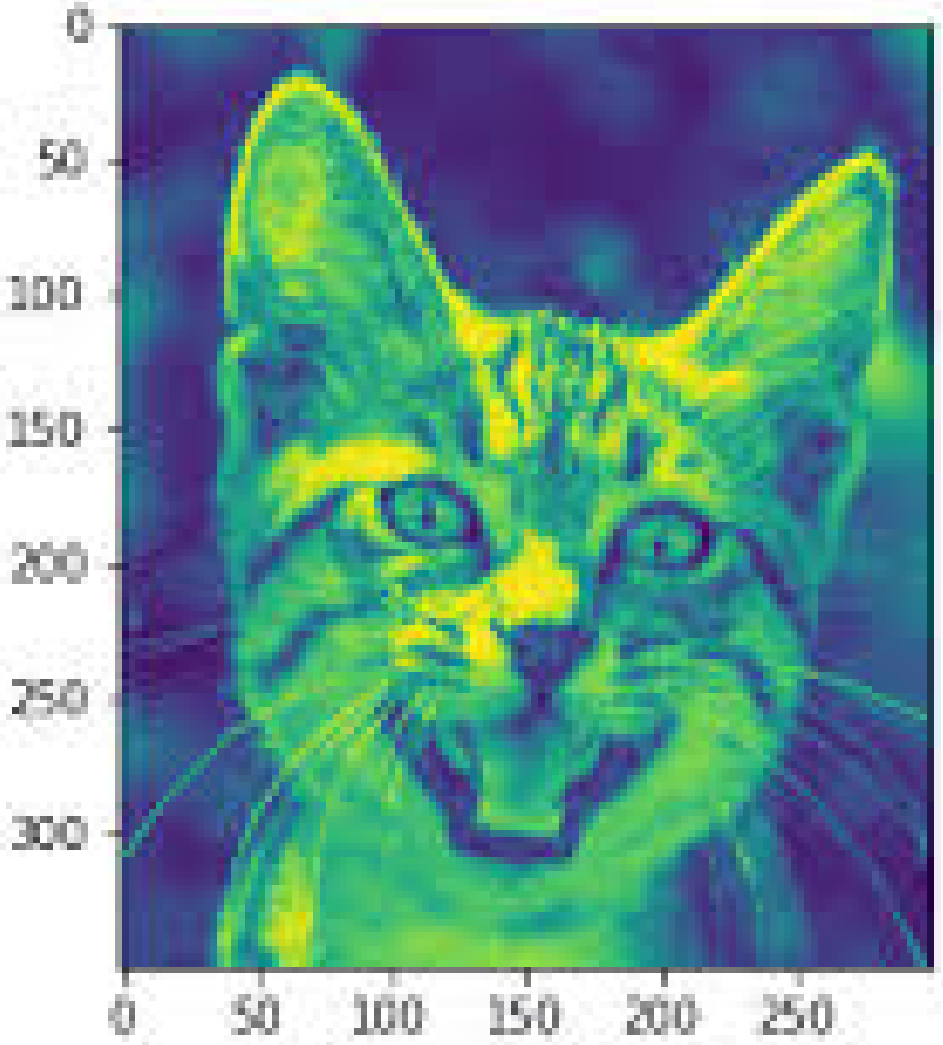

0
50
100
150
200
250
300
0
50
100
150
200
250

Глава 11. Память. DMA & FLASH

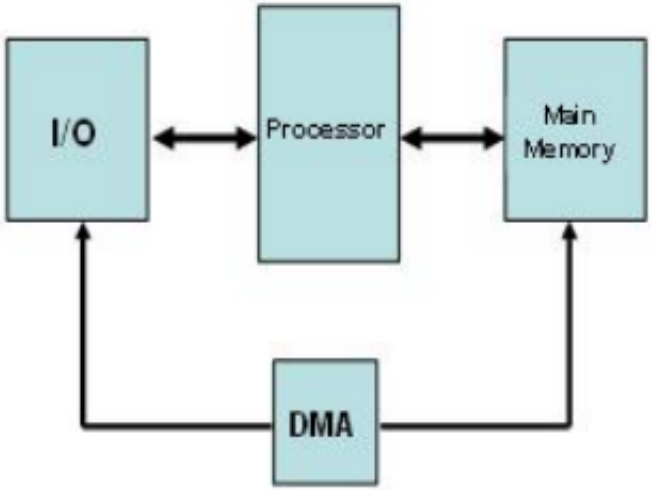
Ежелев Г. И.

25 апреля 2026 г.

DMA

DMA - Direct Memory Access - контроллер прямого доступа к памяти(ПДП).

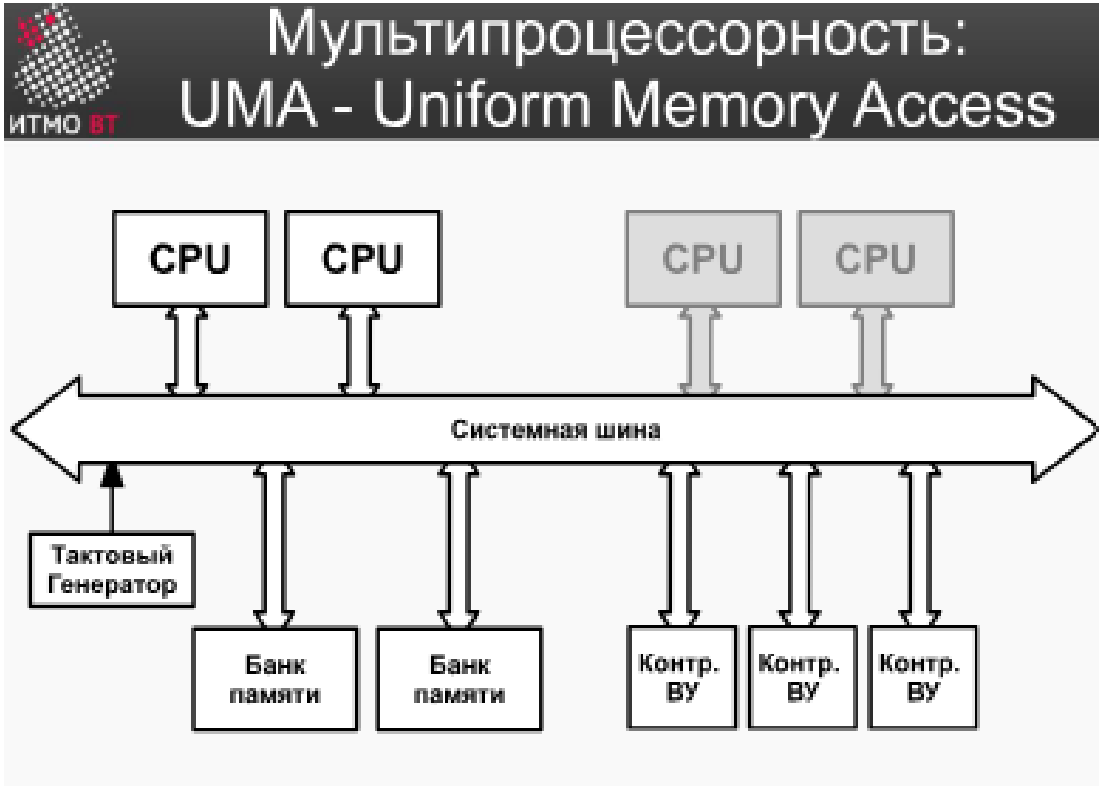
- Что это?
  - Прямой доступ к памяти - это третий из способов взаимодействия с периферией (После прямого чтения и прерываний).
  - Его задача еще больше сократить время взаимодействия процессора с периферией.
  - DMA контроллер напрямую пересылает данные из периферических устройств в память, минуя CPU.



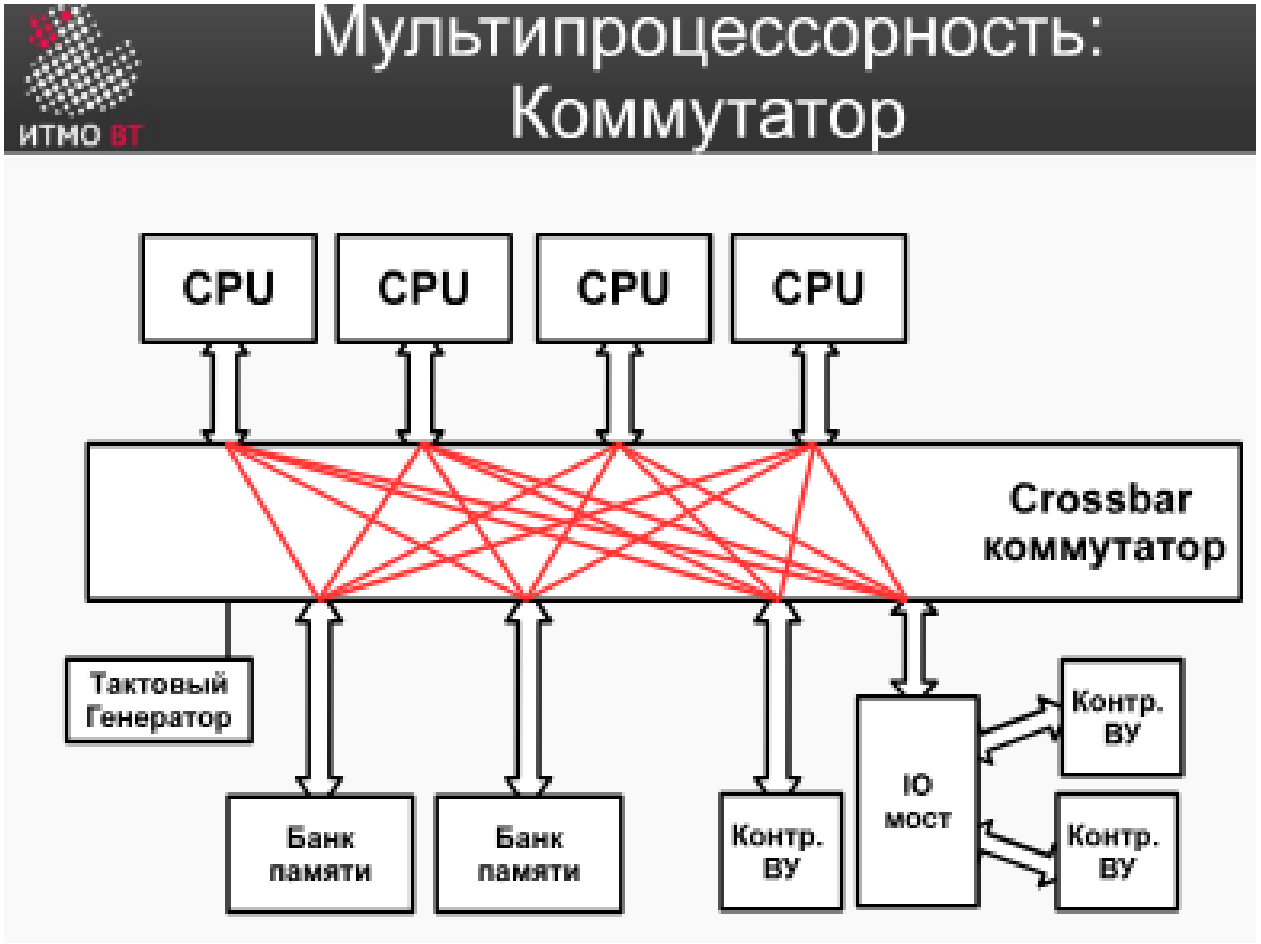
- Отсюда и название - прямой доступ к памяти - ведь по умолчанию все периферические устройства являются ведомыми и не могут сами обращаться к памяти МК (расположенной к слову на тех же шинах AHB, проверьте это на 19 странице datasheet!)

Отступление. Crossbar коммутатор

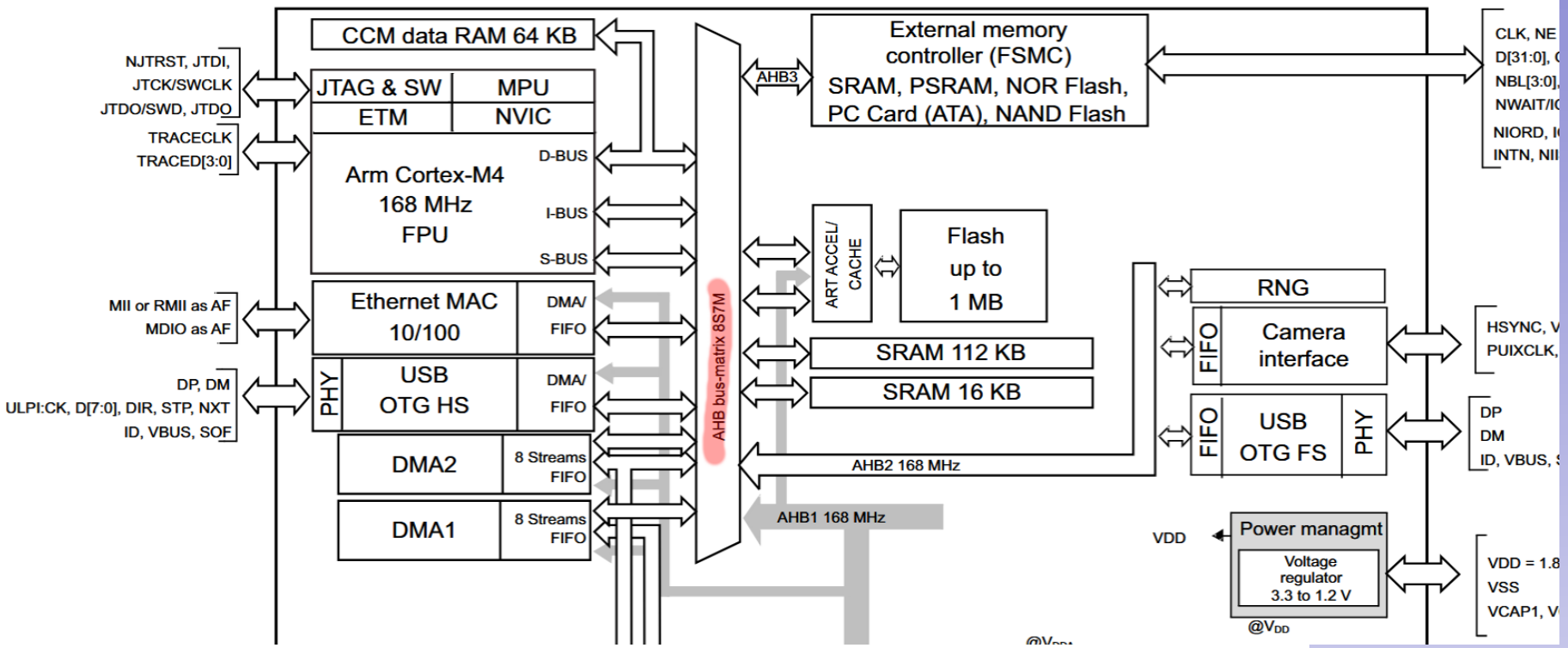
- При подключении нескольких устройств (несколько CPU, DMA и тд) и множества периферических устройств к одной шине вызывает большие задержки (и вообще говоря снижение скорости работы, а не повышение), т.к. в один момент времени шиной может пользоваться только одно устройство



- Решение - Crossbar коммутатор. Он позволяет каждому устройству подключаться к разной периферии одновременно



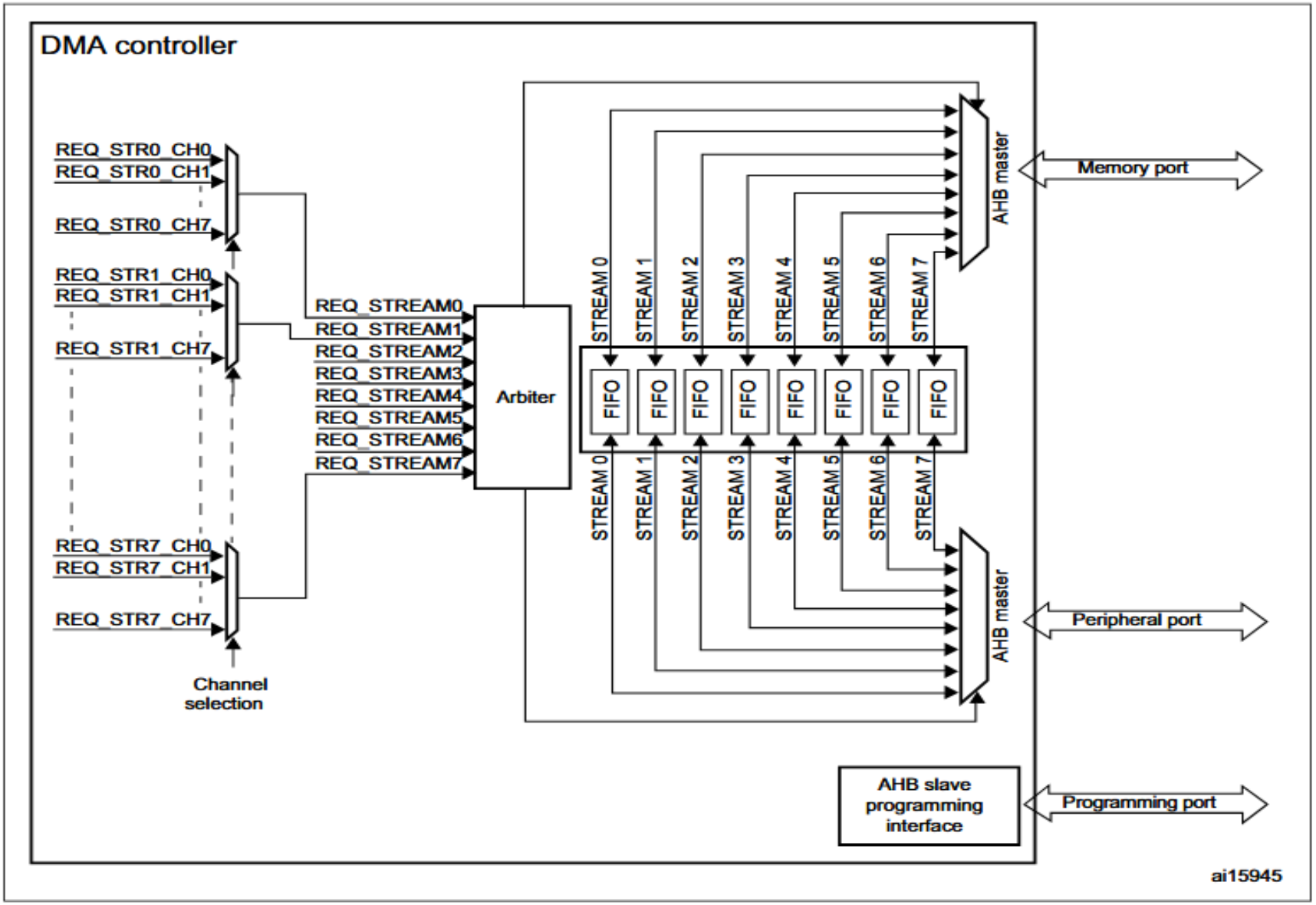
Внутреннее устройство контроллера DMA. AHB Matrix (19стр даташит)



- AHB Matrix - как раз такой коммутатор, позволяющий высокоскоростным устройствам МК пользоваться шинами AHB1,2 почти независимо.
- К ней подключены: ядро; DMA1,2; Ethernet; USB; CCM - Core-Coupled Memory - дополнительная оперативка.

Внутренне устройство DMA

Figure 32. DMA block diagram

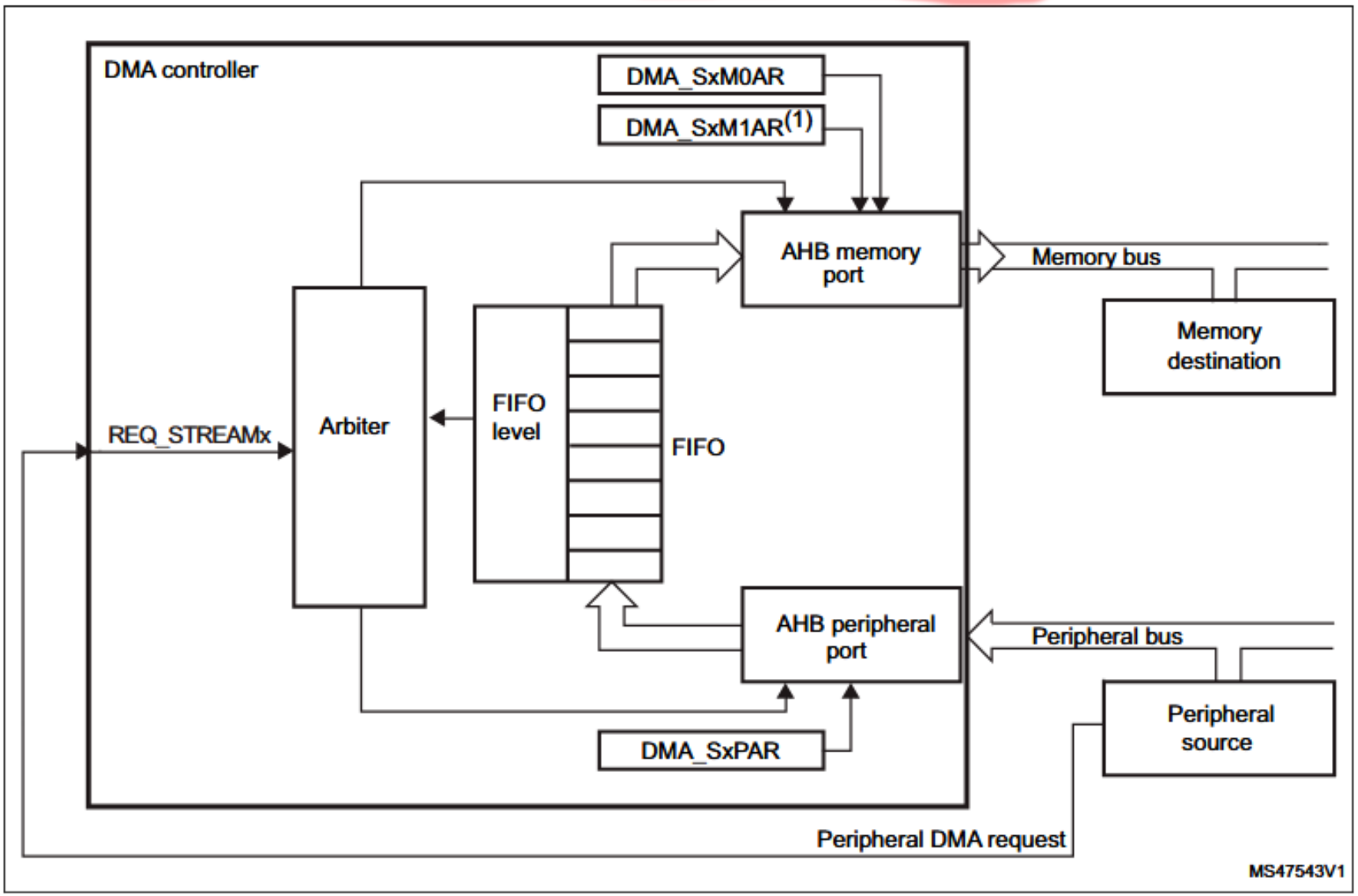


Внутренне устройство DMA

- Каждый контроллер DMA имеет по 8 потоков (stream). Каждый из них может работать с периферией независимо.
- К каждому потоку подключен мультиплексор на выбор одного из 8 каналов передачи (channels)
- Пара: (stream n; channel k) полностью определяет, с каким устройством будет работать DMA.
- Потоки независимы, канал единственен на поток!
- Потоки входят в контроллер DMA(Arbiter). В нем происходит вся логика передачи данных
- Далее Арбитр связывает AHB Memory Port и AHB Peripheral Port с внутренним буфером FIFO.
- Отметим, что буферов FIFO ровно столько, сколько имеется потоков, т.е.8

Арбитр DMA

Figure 36. Peripheral-to-memory mode

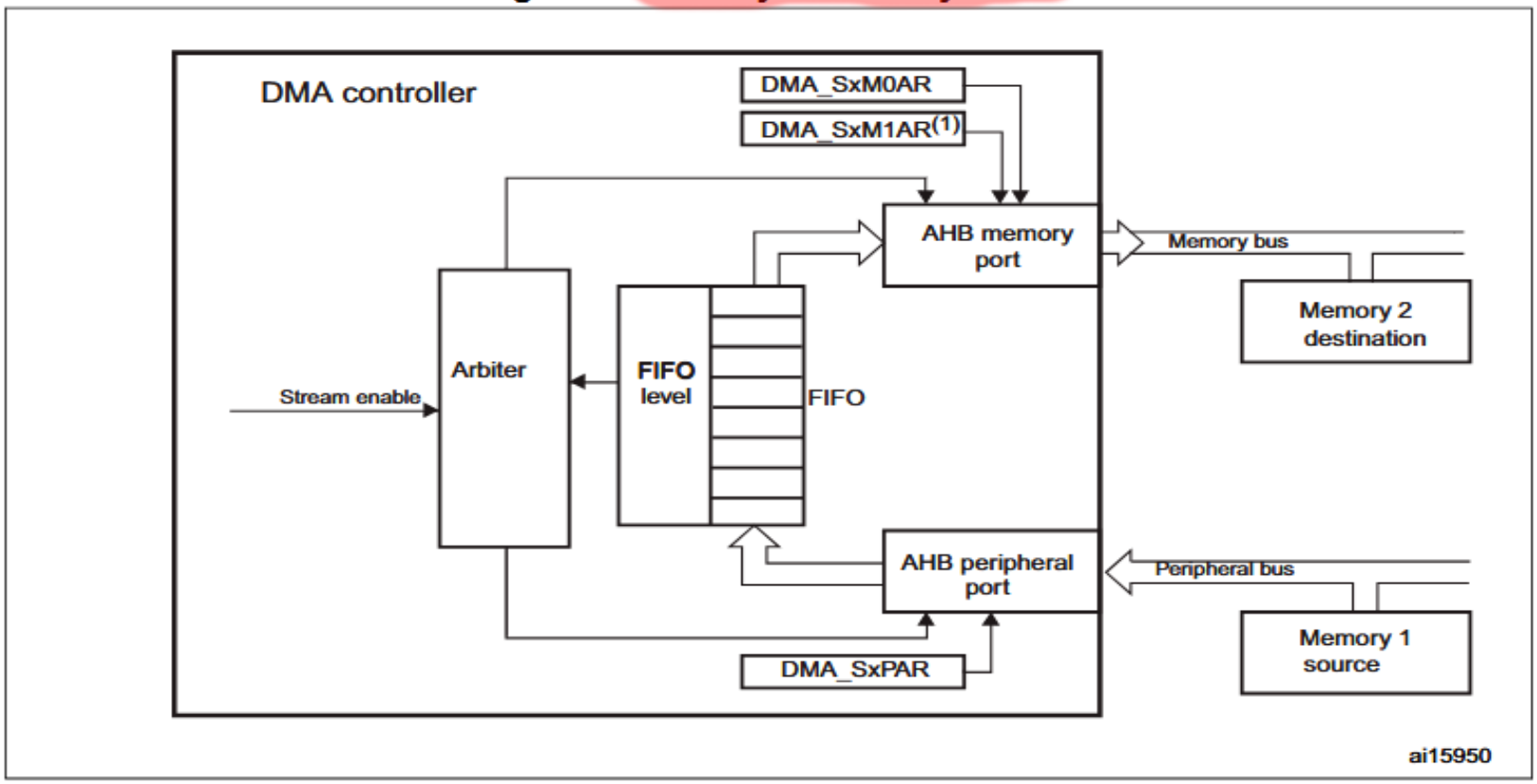


1. For double-buffer mode.

- Рассмотрим самого арбитра изнутри
- Как было видно на первой диаграмме, есть два порта для общения по шинам AHB: Memory port и Peripheral port.
- Они подключены к буферу FIFO - First Input First Output, т.е. он организован по приницпу очереди. Этот буфер используется в случае работы с медленной периферией. Данные накапливаются а далее передаются сразу пачкой для оптимизации использования шины.
- FIFO level - счетчик числа пакетов в FIFO
- Сам арбитр управляет логикой передачи.
- Peripheral source связан с арбитром сигналом запроса передачи данных - REQ_STREAMx
- В другую сторону (Память $\Longrightarrow$ периферия) передача аналогична

- Peripheral Port может использоваться для общения с памятью (в конце концов периферия и память находятся на одной шине и, значит, в одном адресном пространстве). Это нужно для копирования данных в памяти.
  Отметим, что тогда этот порт менее эффективно работает

Figure 38. Memory-to-memory mode

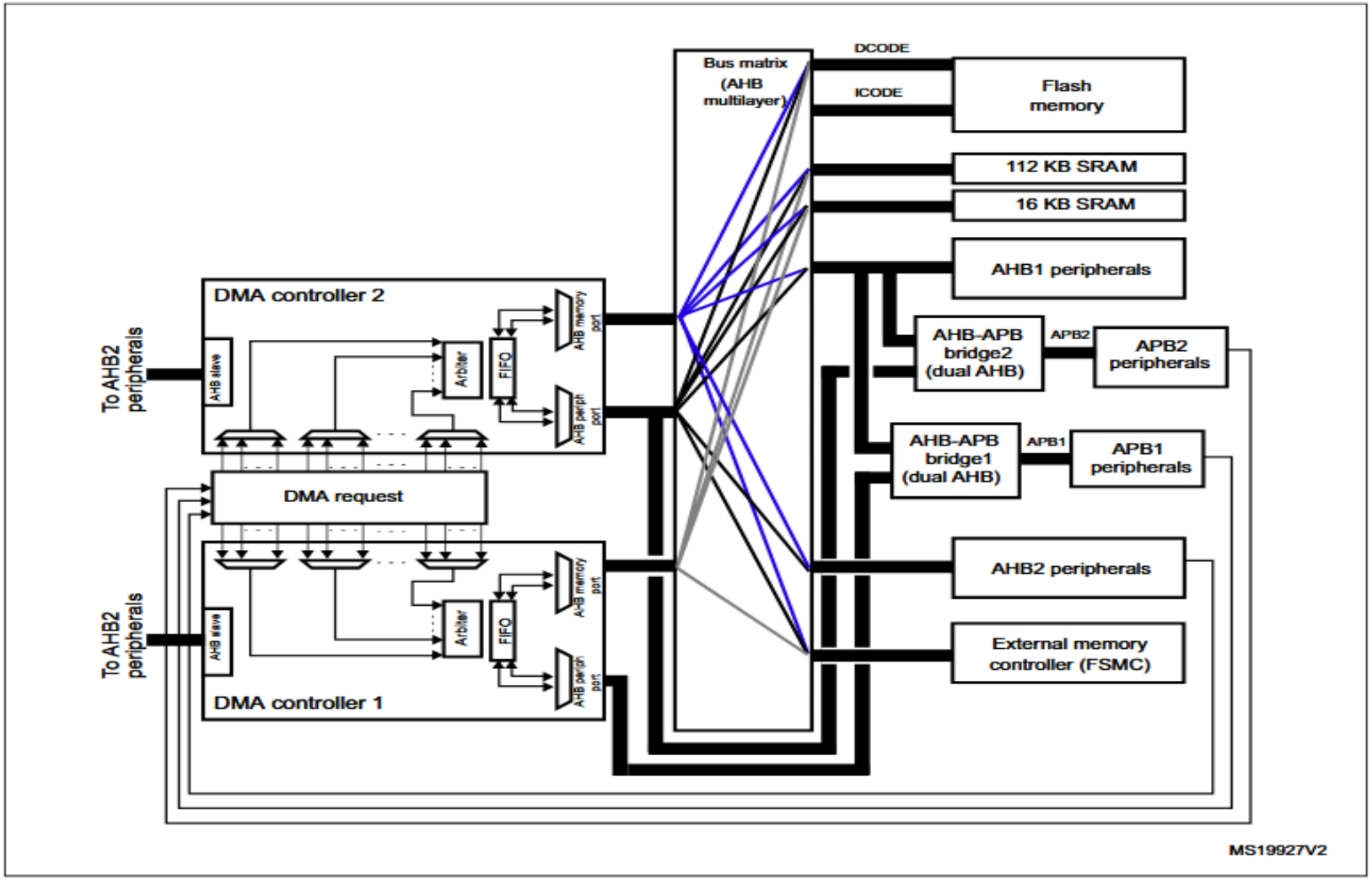


1. For double-buffer mode.

Отличие DMA 1 и 2

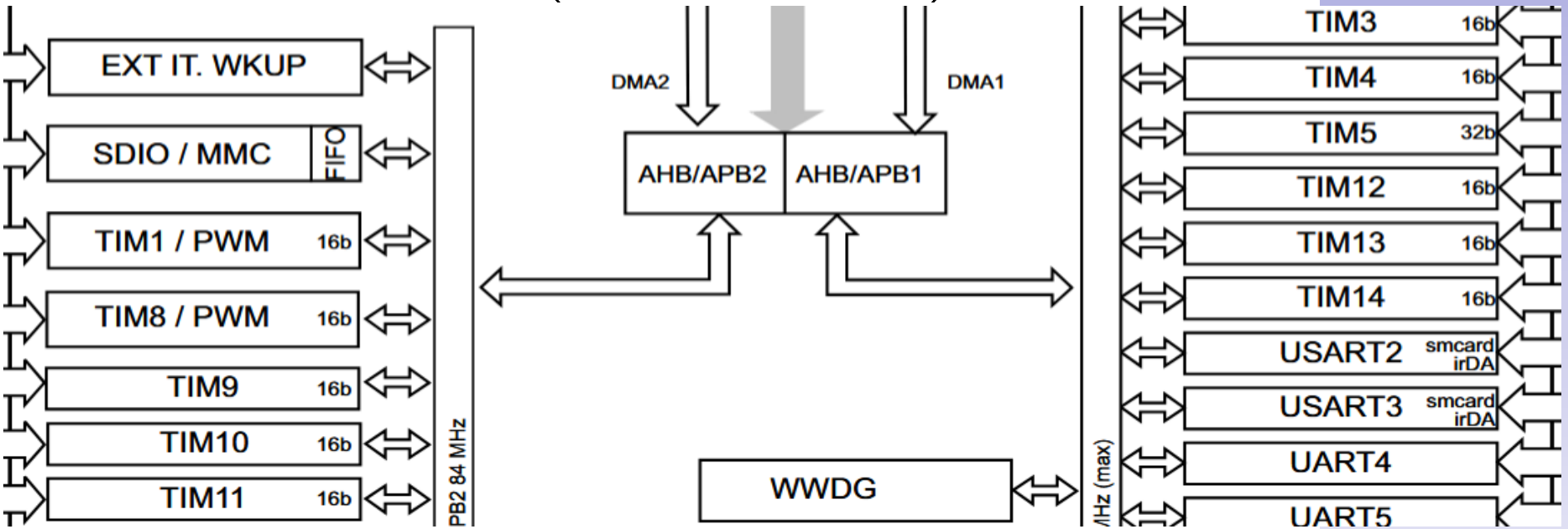
- Единственное отличие двух контроллеров - к чему они подключены внутри матрицы AHB

Figure 33. System implementation of the two DMA controllers (STM32F405xx/07xx and STM32F415xx/17xx)



1. The DMA1 controller AHB peripheral port is not connected to the bus matrix like DMA2 controller. As a result, only DMA2 streams are able to perform memory-to-memory transfers.

• Только у DMA 2 Peripheral port подключен к памяти. Поэтому только DMA2 может заниматься копированием данных в другие сегменты памяти (что и указано в сноске). • Отметим, что у DMA 1 Peripheral Port вообще не подключен к AHB Matrix, а сразу идет на AHB-APB bridge. У DMA 2 он подключен и к матрице, и к мосту. • AHB-APB bridge - узел, соединяющий быструю шину AHB 1 с медленными APB 1, 2 (стр 19 даташит):



Код

Программа лежит в репозитории, ничего нового там нет. Напишем еще один класс.

•Для определения набора (DMA1,2; stream 0-7, channel 0-7) используйте reference manual, главу про DMA, страницы 307-308. Разберем параметры:

- DMA_PeripheralBaseAddr - адрес регистра для взаимодействия (регистры такие же ячейки памяти, так что тоже имеют адрес), например (uint32_t) & USART2->DR
- DMA_Memory0BaseAddr - такой же адрес для памяти
- DMA_DIR - направление передачи данных
- DMA_BufferSize - количество данных для передачи
- DMA_MemoryInc - нужно ли смещать указатель на ячейку порта памяти после каждого пакета. Например удобно при заполнении массива

- DMA_PeripheralInc - то же самое, но на порту периферии (при копировании память $\Longrightarrow$ память это удобно, если же порт подключен к периферии то он будет считывать рандомные регистры в системном порядке).
- DMA_PeripheralDataSize и DMA_MemoryDataSize - размеры передаваемых данных, должны совпадать(быть не меньше чем)с размерами регистров периферии.
- DMA_Mode - режим работы. При режиме Normal - после передачи пакетов в количестве buffer size - останавливает работу. Circular - сбрасывает счетчик (memory inc) и начинает по новой сразу.
- DMA_Priority - приоритет потока внутри DMA.
- DMA_FIFOMode - режим буфера FIFO. Disable - передача каждого пакета сразу. Enable - накопление пакетов в FIFO и передача сразу нескольких.

- DMA_FIFOThreshold - порог заполения буфера FIFO после которого начнется передача
- DMA_MemoryBusrst и DMA_PeripheralBurst - Настройка «взрывной» передачи (только если включен FIFO).
  Если выключен - пакеты передаются последовательно. В промежутках между ними CPU может перехватывать шину.
  INC4,8,16 - передача пакетов пачкой по 4,8,16 штук соответственно.

пример использования

https://github.com/haaroner/Comma_2021/blob/master/src/dma/dma.cpp

https://github.com/haaroner/Comma_2021/blob/master/src/adc/adc.cpp

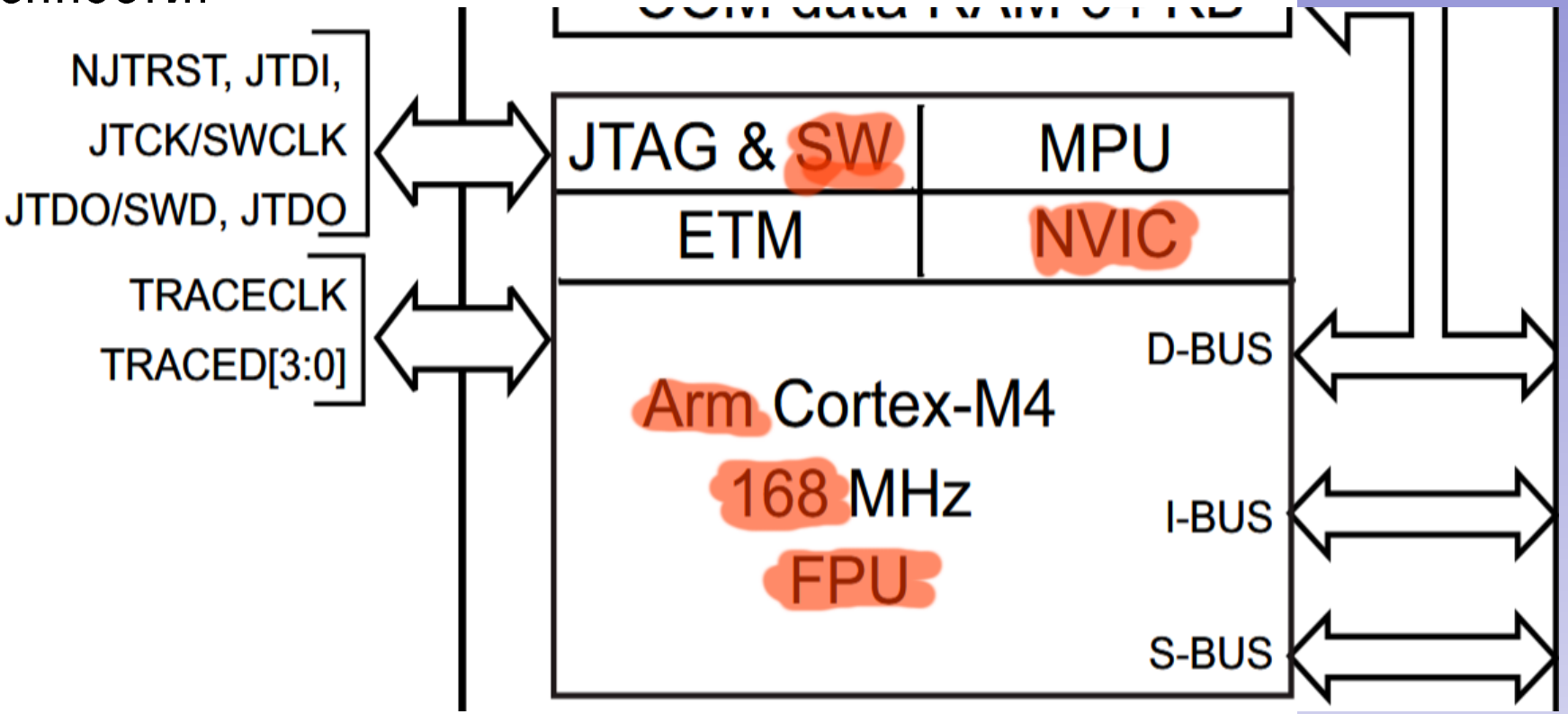
Вызов - в main файле.

Пример для нашего кода(гптшный ☺):

```
uint8_t txBuffer[32] = "Hello DMA from STM32F4!";

int main(void) {
    // На STM32F407 UART2_TX сидит на DMA1, Stream 6, Channel 4
    DMA_Controller uartDma(DMA1_Stream6, DMA_Channel_4);

    // Инициализируем передачу Memory -> Peripheral
    uartDma.InitM2P((uint32_t)txBuffer, (uint32_t)&USART2->DR, 32);

    // Запускаем
    uartDma.Start();

    while (1) {
        // Основной цикл
    }
}
```

Flash-память

•Чип STM32F407 построен на архитектуре V7M - разновидности ARM. Буква **M** - Microcontroller. Особенности:



- Набор инструкций Thumb-2 - использование как компактных 16-битных, так и 32-битных инструкций, для более плотной упаковки программы в памяти.

- MPU - Memory Protection Unit. Модуль для ограничения доступа к различным секторам памяти. Его использования опционально.
- В ядрах Cortex-M4 и старше (есть еще более слабые M3) так же реализованы команды для работы с сигналами DSP - Digital Signal Processing (например элементы быстрого преобразования Фурье), а также появляется FPU - Floating Point Unit.
- В ядрах Cortex-M7 появляется суперскалярность - выполнение нескольких инструкций на одном ядре за такт.

- Полностью аппаратный контроллер прерываний NVIC (Nested Vectored Interrupt Controller) - очень быстрое переключение на прерывания + поддержка хвостовой обработки прерываний (Tail-chaining). Контроллер один раз сохраняет контекст программы и далее обрабатывает все прерывания последовательно (в порядке их приоритетов), не возвращаясь в основную программу, пока все прерывания не будут обработаны.
- Отметим, что без Tail-chaining процессор обрабатывает одно прерывание в конце каждой команды основной программы. Каждая команда выполняется по следующим основным циклам: Command fetch, address fetch, operand fetch, execution, **interrupt**. В конце каждой команды - выполнение ровно одного прерывания. Если сработало 52 прерывания - за 52 команды основной программы они будут обработаны.

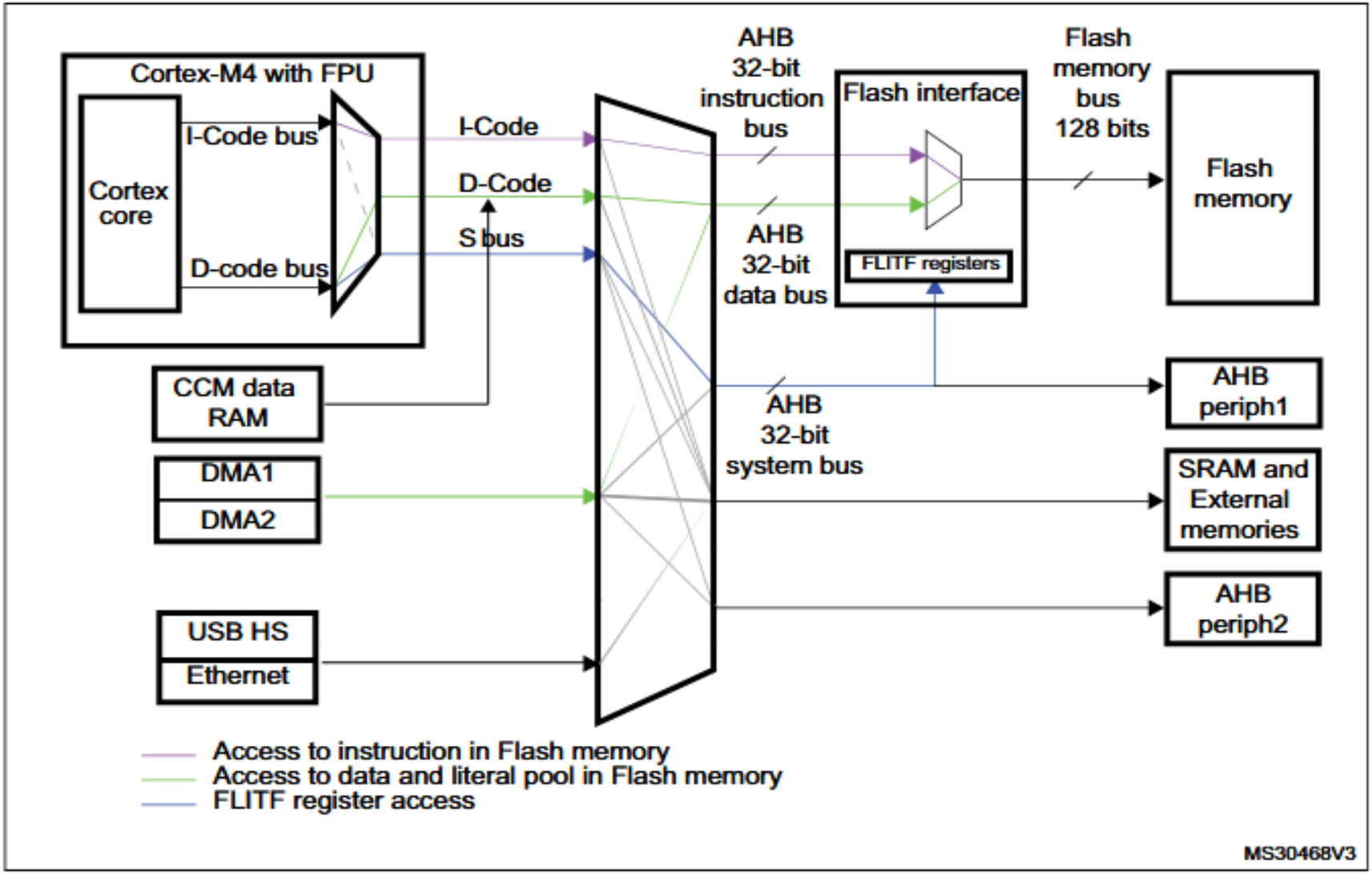
- Flash-память в такой архитектуре имеет неравномерную структуру секторов.

Table 5. Flash module organization (STM32F40x and STM32F41x)

Block	Name	Block base addresses	Size
Main memory	Sector 0	0x0800 0000 - 0x0800 3FFF	16 Kbytes
	Sector 1	0x0800 4000 - 0x0800 7FFF	16 Kbytes
	Sector 2	0x0800 8000 - 0x0800 BFFF	16 Kbytes
	Sector 3	0x0800 C000 - 0x0800 FFFF	16 Kbytes
	Sector 4	0x0801 0000 - 0x0801 FFFF	64 Kbytes
	Sector 5	0x0802 0000 - 0x0803 FFFF	128 Kbytes
	Sector 6	0x0804 0000 - 0x0805 FFFF	128 Kbytes
	. . .	. . .	. . .
	Sector 11	0x080E 0000 - 0x080F FFFF	128 Kbytes
System memory		0x1FFF 0000 - 0x1FFF 77FF	30 Kbytes
OTP area		0x1FFF 7800 - 0x1FFF 7A0F	528 bytes
Option bytes		0x1FFF C000 - 0x1FFF C00F	16 bytes

Flash-память. Устройство

Figure 3. Flash memory interface connection inside system architecture (STM32F405xx/07xx and STM32F415xx/17xx)



Интерфейс ART-Accelerator

Вся программа лежит во Flash-памяти. Есть одна загвостка - время доступа к ней: 5 циклов ожидания (wait states). Без ускорителя процессор бы простаивал 80% времени, ожидая следующую инструкцию. Поэтому Flash-память подключена к МК по интерфейсу ART-Accelerator - Adaptive Real-Time Accelerator. Как он работает:

- ART - это продвинутая система кэширования, оптимизированная под работу именно с кодом, а не данными
- Упреждающее чтение (Instruction Prefetch) - Flash в STM32F407 имеет ширину шины 128 бит. За одно обращение к памяти контроллер считывает сразу 4 инструкции по 32 бита (или 8 инструкций по 16 бит). Пока процессор выполняет первую инструкцию, остальные три уже лежат в очереди.

- Кэш инструкций (Instruction Cache) - При выполнении циклов ART запоминает, какие инструкции ему встретились в течение итерации и кэширует их для следующего витка цикла. В случае ветвлений ART пытается предсказать, какая из веток программы (if/else) будет выполнена и заранее кэширует инструкции по этой ветке.
- Кэш данных (Data Cache) - Аналогично инструкциям, константы и массивы, хранящиеся во Flash, кэшируются для быстрого доступа.
- ART Accelerator ускоряет только чтение. Операции записи и стирания всё так же требуют времени и блокируют доступ к Flash.

Интерфейс ART-Accelerator. Код


```
986
987        // Настраиваем режим чтения Flash-памяти
988        FLASH->ACR |= FLASH_ACR_ICEN | FLASH_ACR_DCEN;
989        FLASH->ACR &= ~FLASH_ACR_LATENCY;
990        FLASH->ACR |= FLASH_ACR_LATENCY_5WS;
991
```

В файле system_stm32f4xx.c включаются все эти функции кроме instruction prefetch.
Для этого нужно модифицировать 1 строку:
FLASH->ACR |= FLASH_ACR_ICEN | FLASH_ACR_DCEN | FLASH_ACR_PRFTEN;
Я сам не проверял, не сломается ли всё от этого.

Важная особенность Flash-памяти: её жизненный цикл - около 10 000 циклов стирания.
Для записи программы и хранения калибровочных данных - хватает. Для логирования и постоянного использования в цикле - точно нет.

Flash. Код

Особенности работы:

- Стирание - нельзя просто перезаписывать данные: переход бита $1 \rightarrow 0$ необратим. Только стирание сектора возвращает его в исходное состояние (все биты 1).
- Запрет на запись/чтение: при попытке писать в тот же банк памяти, из которого выполняется код, процессор замирает (stall) до завершения операции.
- Так же вы сами можете открывать и блокировать сектора для записи в них.
- От напряжения зависит размер записываемых данных за раз: в случае 3.3 вольт - это 32 бита. Если вы попытаетесь это превысить - может подняться флаг ошибки PGAERR (Programming Alignment Error) в регистре FLASH_SR.
- При низком напряжении (например, 1.8В вместо 3.3В) контроллер физически не может обеспечить

достаточный ток/заряд для надежного пробоя диэлектрика затвора во Flash-памяти сразу для 32 бит.

- Запись может произойти частично (не все биты перейдут из 1 в 0). Вы получите недетерминированное состояние ячейки. При чтении она может возвращать разные значения или вызывать ошибку ECC (Error Correction Code).

Flash. Код


```c
#pragma once

#include "project_config.h"

#define STANDART_PAGE_WRITING_ADDRESS 0x080E0000

void write_to_FLASH(uint8_t _data)
{
  FLASH_Unlock();
  FLASH_EraseSector(FLASH_Sector_11, VoltageRange_3);
  FLASH_ProgramWord(STANDART_PAGE_WRITING_ADDRESS, _data);
  FLASH_Lock();
}

uint32_t read_from_FLASH()
{
  FLASH_Unlock();
  return *(__IO uint32_t*)STANDART_PAGE_WRITING_ADDRESS;
  FLASH_Lock();
}
```